

CS3300 - Tools of the Trade #3

Student Demo Notes

jQuery | APIs | AJAX | RESTful APIs

Big picture: We are creating a webpage using HTML + CSS + JavaScript. jQuery helps JavaScript interact with the webpage -> APIs let our application interact with another application/service -> HTTP defines how requests and responses travel -> AJAX lets the browser make those requests without reloading the page -> REST provides an architectural style for organizing an API around resources.

Demo 1: jQuery

Goal: Use jQuery to select elements on a webpage and respond to user events.

1. Create a basic HTML page. In VS Code, you can type ! and press Tab/Enter to create the HTML boilerplate.
2. Include the jQuery library and your client-side script.js file.

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
<script src="script.js"></script>
```

3. Add a hidden-looking heading and a button to the body.

```
<h1 style="text-align:center; color:white;" id="hidden">I was hidden!</h1>
<button id="button1">Alert me!</button>
```

4. In script.js, wait until the document is ready. Use jQuery to change the heading color on hover, hide it on double-click, and respond to the button click.

```
$(document).ready(function() {
  $("#hidden").hover(
    function() { $(this).css("color", "black"); },
    function() { $(this).css("color", "white"); }
  );

  $("#hidden").dblclick(function() {
    $(this).hide();
  });

  $("#button1").click(function() {
    alert("I am alerting you");
  });
});
```

- Key idea: \$("#hidden") returns a jQuery object wrapping the matching DOM element. jQuery provides concise methods such as .hover(), .dblclick(), .hide(), and .css().

Demo 2: Calling an API

Goal: See how an application requests data from another service through an API.

5. Use the weather API demonstrated in class.

6. An API call uses an endpoint and may include parameters. The service may require an API key to authenticate the caller.
- The response is represented as data; in this demo the format is JSON.
- A ? begins query parameters in a URL and & separates additional parameters.
- Changing a parameter such as units changes what the service returns (for example, temperature units).

Key idea: An API is the boundary through which one piece of software asks another application/service to do something or provide information.

Demo 3: AJAX

Goal: Have JavaScript make an HTTP request behind the scenes and receive JSON data without reloading the webpage.

7. For this demo, JSONBin acts as a simple server containing JSON data.

```
{
  "name": "cs 3300",
  "company": "Georgia Tech",
  "semester": "Fall 2022",
  "num_students": 40
}
```

8. Create a simple webpage with a button and include jQuery and script.js.

```
<button id="b1">Click me!</button>
```

9. In script.js, send a GET request when the button is clicked.

```
$(document).ready(function() {
  $("#b1").click(function() {
    $.ajax({
      url: "YOUR_JSONBIN_URL",
      type: "GET",
      dataType: "json",
      timeout: 2500,
      success: function(result) {
        console.log("Completed with success and data:", result);
      },
      error: function(result) {
        console.log("complete with error");
      },
      complete: function() {
        console.log("complete");
      }
    });
  });
});
```

- success runs when the request succeeds; error runs when it fails; complete runs when the request finishes.
- For private data, the request may also require an authentication header/API key, as demonstrated in class.

Key idea: In a real application, the user does not type an API URL. Client-side JavaScript sends the request, receives the response, and can use the returned data to update the webpage.

Demo 4: Building a REST API with Node.js and Express

Goal: Build a server-side API organized around resources and HTTP methods.

- Node.js allows JavaScript to execute outside the browser.
- Express is a framework for building a web server/API with Node.js.

10. Initialize a Node project and install Express.

```
npm init
npm i express
```

- The class demo also introduced Joi for validation and nodemon for automatically restarting a Node application when files change.

```
npm i joi
npm i -g nodemon
```

11. Create server-side JavaScript and initialize some data so that the API has a resource to work with.

```
const students = [
  { name: 'George', id: 1 },
  { name: 'Sam', id: 2 },
  { name: 'Mary', id: 3 },
  { name: 'Rebecca', id: 4 },
  { name: 'Tanya', id: 5 }
];
```

12. Implement REST endpoints for the students resource. The class demo exercises these operations:

```
GET    /api/students      -> retrieve all students
GET    /api/students/:id   -> retrieve one student
POST   /api/students      -> create a student
PUT    /api/students/:id  -> update a student
DELETE /api/students/:id   -> delete a student
```

- GET retrieves a resource, POST creates a resource, PUT updates an existing resource, and DELETE removes a resource.
- The resource stays on the server; the server sends a representation of its current state to the client, typically as JSON.

Component	Runs where?	Responsibility
index.html	Browser	Provides a basic UI for the user.
public/script.js	Browser	Responds to UI actions and sends GET/POST/PUT/DELETE requests to the REST API.
server.js	Node.js	Implements the REST endpoints and manages product data.

```
GET    /api/products      -> display all products
GET    /api/products/:id   -> display a specific product
POST   /api/products      -> create a new product
PUT    /api/products/:id  -> update an existing product
DELETE /api/products/:id   -> delete a product
```

What You Should Be Able to Explain After the Demos

- The difference between JavaScript running in the browser and JavaScript running through Node.js on the server.

- How jQuery helps JavaScript interact with the DOM.
- What an API is and how HTTP requests and responses cross the client-server boundary.
- How AJAX allows client-side JavaScript to make requests without reloading the whole page.
- How REST organizes an API around resources and HTTP methods.
- How GET, POST, PUT, and DELETE correspond to read, create, update, and delete operations.